Design a C++ program for a Smart Farm Management System. The system should manage data related to different types of crops and their cultivation details. It should support recording, displaying, and analyzing data to assist farmers in making decisions.

Your program must:

1. Define a base class Crop with attributes like name, area, season, and yieldPerAcre. Use appropriate access specifiers and constructors.

2.Derive two classes CereatCrop and VegetableCrop from Crop using inheritance, adding their specific data members (e.g., fertilizer type, irrigation frequency).

3. Use constructor overloading and a virtual destructor.

4.Apply runtime polymorphism to calculate and display total yield for both crop types.

5. Demonstrate operator overloading to compare two crops based on yield.

6.Create an abstract class Advisory that has a pure virtual function giveAdvice().Implement this in derived classes to provide farming tips for different crops.

7. Use file handling to save and retrieve crop data.

8. Include at least one use of static members (e.g., count of crops).

9. Ensure appropriate input/output formatting and proper exception handling (e.g., invalid area or yield input).

Algorithm: Smart Farm Management System in C++

Step 1: Start the program.

Step 2: Define a base class Crop with data members such as crop name, area, season and yield per acre.

Step 3: Create default and parameterized constructors to initialize crop details. Declare a virtual destructor and a static variable to count crop objects.

Step 4: Define a virtual function totalYield() to calculate the total crop yield by multiplying area with yield per acre.

Step 5: Overload the greater-than (>) operator to compare two crops based on their total yield.

Step 6: Derive two classes, CerealCrop and VegetableCrop, from the base class Crop.

- CerealCrop stores fertilizer type.
- VegetableCrop stores irrigation frequency.

Step 7: Create an abstract class Advisory containing a pure virtual function giveAdvice().

Step 8: Implement the giveAdvice() function in CerealAdvisory and VegetableAdvisory classes to provide crop-specific farming suggestions.

Step 9: Create a menu-driven system with the following options:

1. Add cereal crop.
2. Add vegetable crop.
3. Display all crop details.
4. Compare two crops.
5. Display farming advisory.
6. Save crop data to a file.
7. Retrieve crop data from a file.
8. Display total crop count.
9. Exit.

Step 10: Accept crop details from the user and validate the entered area, yield and irrigation frequency.

Step 11: Create crop objects using constructors and store them in a vector of base-class pointers.

Step 12: Display crop details and calculate total yield using runtime polymorphism.

Step 13: Compare two crop objects using the overloaded greater-than operator and display the crop with higher yield.

Step 14: Display suitable farming advice using the abstract class and its derived classes.

Step 15: Use file handling (ofstream) to save crop information in farmdata.txt.

Step 16: Use file handling (ifstream) to retrieve previously stored crop information.

Step 17: Use the static member function to display the total number of crop objects.

Step 18: Handle invalid inputs and runtime errors using try, throw and catch.

Step 19: Repeat the menu until the user selects Exit (option 9).

Step 20: Stop the program.

```
#include <iostream>
#include <fstream>
#include <vector>
#include <memory>
#include <string>
#include <sstream>
```

```
#include <limits>
using namespace std;

class Crop {
protected:
    string name, season;
    double area, yieldPerAcre;
    static int count;
public:
    Crop() {
        name = "Unknown";
        season = "Unknown";
        area = 0;
        yieldPerAcre = 0;
        count++;
    }
    Crop(string n, double a, string s, double y) {
        name = n;
        area = a;
        season = s;
        yieldPerAcre = y;
        count++;
    }
    virtual ~Crop() {
        count--;
    }
    virtual double totalYield() const {
        return area * yieldPerAcre;
    }
    virtual void display() const {
```

```
        cout << "\nCrop Name: " << name;
        cout << "\nArea: " << area << " acres";
        cout << "\nSeason: " << season;
        cout << "\nYield Per Acre: " << yieldPerAcre;
        cout << "\nTotal Yield: " << totalYield() << endl;
    }
    bool operator>(const Crop &c) const {
        return totalYield() > c.totalYield();
    }
    static int getCount() {
        return count;
    }
    string getName() const {
        return name;
    }
    double getArea() const {
        return area;
    }
    string getSeason() const {
        return season;
    }
    double getYield() const {
        return yieldPerAcre;
    }
};
int Crop::count = 0;

class CerealCrop : public Crop {
    string fertilizerType;
public:
```

```
    CerealCrop() : Crop() {
        fertilizerType = "Not specified";
    }
    CerealCrop(string n, double a, string s, double y, string f)
        : Crop(n, a, s, y) {
        fertilizerType = f;
    }
    void display() const override {
        cout << "\n--- Cereal Crop Details ---";
        Crop::display();
        cout << "Fertilizer Type: " << fertilizerType << endl;
    }
    string getFertilizer() const {
        return fertilizerType;
    }
};

class VegetableCrop : public Crop {
    int irrigationFrequency;
public:
    VegetableCrop() : Crop() {
        irrigationFrequency = 0;
    }
    VegetableCrop(string n, double a, string s, double y, int i)
        : Crop(n, a, s, y) {
        irrigationFrequency = i;
    }
    void display() const override {
        cout << "\n--- Vegetable Crop Details ---";
        Crop::display();
```

```
        cout << "Irrigation Frequency: " << irrigationFrequency << " times/week" << endl;
    }
    int getIrrigation() const {
        return irrigationFrequency;
    }
};

class Advisory {
public:
    virtual void giveAdvice() = 0;
    virtual ~Advisory() {}
};

class CerealAdvisory : public Advisory {
public:
    void giveAdvice() override {
        cout << "\nCereal Farming Advice:";
        cout << "\n1. Use good quality seeds.";
        cout << "\n2. Apply fertilizers properly.";
        cout << "\n3. Monitor soil nutrients regularly.";
        cout << "\n4. Control weeds and pests.\n";
    }
};
class VegetableAdvisory : public Advisory {
public:
    void giveAdvice() override {
        cout << "\nVegetable Farming Advice:";
        cout << "\n1. Maintain proper irrigation.";
        cout << "\n2. Use organic manure.";
        cout << "\n3. Remove weeds regularly.";
```

```
        cout << "\n4. Monitor plants for diseases.\n";
    }
};

int main() {
    vector<unique_ptr<Crop>> crops;
    int choice;
    do {
        cout << "\n==================================";
        cout << "\n     SMART FARM MANAGEMENT";
        cout << "\n==================================";
        cout << "\n1. Add Cereal Crop";
        cout << "\n2. Add Vegetable Crop";
        cout << "\n3. Display All Crops";
        cout << "\n4. Compare Two Crops";
        cout << "\n5. Farming Advisory";
        cout << "\n6. Save Crop Data";
        cout << "\n7. Retrieve Crop Data";
        cout << "\n8. Display Total Crop Count";
        cout << "\n9. Exit";
        cout << "\nEnter your choice: ";
        cin >> choice;
        try {
            if (cin.fail())
                throw runtime_error("Invalid menu input!");
            if (choice == 1 || choice == 2) {
                string name, season;
                double area, yield;
                cout << "Enter Crop Name: ";
                cin >> ws;
```

```
            getline(cin, name);
            cout << "Enter Area (acres): ";
            cin >> area;
            if (cin.fail() || area <= 0)
                throw invalid_argument("Invalid area!");
            cout << "Enter Season: ";
            getline(cin >> ws, season);
            cout << "Enter Yield Per Acre: ";
            cin >> yield;
            if (cin.fail() || yield <= 0)
                throw invalid_argument("Invalid yield!");
            if (choice == 1) {
                string fertilizer;
                cout << "Enter Fertilizer Type: ";
                cin >> ws;
                getline(cin, fertilizer);
                crops.push_back(make_unique<CerealCrop>(name, area, season, yield,
fertilizer));
            } else {
                int irrigation;
                cout << "Enter Irrigation Frequency (times per week): ";
                cin >> irrigation;
                if (cin.fail() || irrigation < 0)
                    throw invalid_argument("Invalid irrigation frequency!");
                crops.push_back(make_unique<VegetableCrop>(name, area, season, yield,
irrigation));
            }
            cout << "\nCrop Added Successfully!\n";
        } else if (choice == 3) {
            if (crops.empty())
                cout << "\nNo crop records available.\n";
```

```
    for (const auto &c : crops) {
        c->display();
        cout << "-------------------------\n";
    }
} else if (choice == 4) {
    if (crops.size() < 2) {
        cout << "\nAdd at least two crops first.\n";
    } else {
        int a, b;
        cout << "Enter first crop number (1 to " << crops.size() << "): ";
        cin >> a;
        cout << "Enter second crop number (1 to " << crops.size() << "): ";
        cin >> b;
        if (cin.fail() || a < 1 || b < 1 || a > crops.size() || b > crops.size())
            throw out_of_range("Invalid crop number!");
        if (crops[a-1]->operator>(*crops[b-1]))
            cout << crops[a-1]->getName() << " has higher total yield.\n";
        else if (crops[b-1]->operator>(*crops[a-1]))
            cout << crops[b-1]->getName() << " has higher total yield.\n";
        else
            cout << "Both crops have equal yield.\n";
    }
} else if (choice == 5) {
    int type;
    cout << "\n1. Cereal Crop Advice";
    cout << "\n2. Vegetable Crop Advice";
    cout << "\nEnter choice: ";
    cin >> type;
    if (type == 1) {
        CerealAdvisory a;
```

```
            Advisory *ptr = &a;
            ptr->giveAdvice();
        } else if (type == 2) {
            VegetableAdvisory a;
            Advisory *ptr = &a;
            ptr->giveAdvice();
        } else {
            throw invalid_argument("Invalid advisory choice!");
        }
    } else if (choice == 6) {
        ofstream file("farmdata.txt");
        if (!file)
            throw runtime_error("Unable to open file!");
        for (const auto &c : crops) {
            if (auto p = dynamic_cast<CerealCrop*>(c.get())) {
                file << "Cereal|" << p->getName() << "|" << p->getArea() << "|" << p->getSeason() << "|" << p->getYield() << "|" << p->getFertilizer() << endl;
            } else if (auto p = dynamic_cast<VegetableCrop*>(c.get())) {
                file << "Vegetable|" << p->getName() << "|" << p->getArea() << "|" << p->getSeason() << "|" << p->getYield() << "|" << p->getIrrigation() << endl;
            }
        }
        file.close();
        cout << "\nCrop Data Saved Successfully!\n";
    } else if (choice == 7) {
        ifstream file("farmdata.txt");
        if (!file)
            throw runtime_error("File not found!");
        crops.clear();
        string line;
        while (getline(file, line)) {
```

```
                stringstream ss(line);
                string type, name, season, extra;
                double area, yield;
                getline(ss, type, '|');
                getline(ss, name, '|');
                ss >> area;
                ss.ignore();
                getline(ss, season, '|');
                ss >> yield;
                ss.ignore();
                getline(ss, extra);
                if (type == "Cereal") {
                    crops.push_back(make_unique<CerealCrop>(name, area, season, yield,
extra));
                } else if (type == "Vegetable") {
                    crops.push_back(make_unique<VegetableCrop>(name, area, season, yield,
stoi(extra)));
                }
            }
            file.close();
            cout << "\nCrop Data Retrieved Successfully!\n";
        } else if (choice == 8) {
            cout << "\nTotal Number of Crops: " << Crop::getCount() << endl;
        } else if (choice == 9) {
            cout << "\nExiting Smart Farm Management System.\n";
        } else {
            throw invalid_argument("Invalid menu choice!");
        }
    } catch (const exception &e) {
        cout << "\nError: " << e.what() << endl;
        if (cin.fail()) {
```

```
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
        }
    }
  } while (choice != 9);
  return 0;
}
```

OUTPUT

==========

SMART FARM MANAGEMENT

==========

1. Add Cereal Crop
2. Add Vegetable Crop
3. Display All Crops
4. Compare Two Crops
5. Farming Advisory
6. Save Crop Data
7. Retrieve Crop Data
8. Display Total Crop Count
9. Exit

Enter your choice: 5

1. Cereal Crop Advice
2. Vegetable Crop Advice

Enter choice: 2

Vegetable Farming Advice:

1. Maintain proper irrigation.
2. Use organic manure.
3. Remove weeds regularly.
4. Monitor plants for diseases.